TAGENT: A TEACHER-AGENT SYSTEM FOR LEARNING EFFECTIVE PROGRAMMING FEEDBACK

François Chesnay & Andrew Nichols
Stanford University
Stanford, CA 94305, USA
{fchesnay, an160}@stanford.edu

1 PROBLEM SPECIFICATION AND RESEARCH QUESTIONS

Tagent is a teacher-agent system that investigates whether the persistent memory of prior teaching interactions can improve feedback to students repairing incorrect code, without revealing solutions. The problem is that existing approaches to programming assistance have important limitations. Autograders identify whether code passes or fails tests but provide limited diagnostic guidance. Human teaching assistants can offer individualized feedback that helps students understand and correct their mistakes, but such assistance does not scale easily. Automated code-repair systems that provide corrected solutions may bypass the student's problem-solving process. The challenge is to provide effective, scalable feedback that helps students identify and repair errors while preserving their responsibility for solving the problem. To investigate whether persistent teacher experience can help address this challenge, we pose the following overarching research question: *Can a teacher agent use information from prior teaching interactions to improve the effectiveness of its feedback without directly providing the solution?* In phase A1 of our project, we hold the student large language model (LLM) fixed to isolate the teacher's contribution, and investigate three supporting questions: **RQ1 (Memory Effect):** Does equipping the teacher with Persistent Teacher Experience Memory (PTEM) improve teaching outcomes relative to an otherwise equivalent no-memory teacher? **RQ2 (Memory Ablation):** Which components of PTEM contribute to its effect on teaching outcomes? **RQ3 (Human Validation):** Do human evaluators find feedback from the memory-equipped teacher more helpful than feedback from the no-memory teacher?

2 ENVIRONMENT & DATA

2.1 PROGRAMMING ENVIRONMENT AND TOOLS

The Tagent A1 environment uses Python and Harbor for executing and testing student programs. Harbor, configured to use a Docker-based sandbox, provides the execution and verification infrastructure for evaluating candidate repairs against executable tests. The proposed system supports iterative execution, returning test outcomes after each student revision until the program passes or the permitted repair-round budget is exhausted. Interaction traces (recorded in Agent Trajectory Interchange Format (ATIF)) and resource measurements, including model calls, token usage, latency, and cost, are intended to support evaluation, reproducibility, and failure analysis.

2.2 DATASET CONSTRUCTION AND PARTITIONING

The experiments use Python 3 submissions, problem statements and official tests from HuggingFace `open-r1/codeforces`. A student attempt starts from a rejected submission. The stored tables also keep the accepted submissions. The working copies are in the Parquet data format.

The source release contains 10,024 problems. Keeping problems that have a Python 3 submission leaves 6,038. A problem is then kept only when its official tests are complete, it has both an accepted and a rejected Python 3 submission, no official test expects more than 1 MB of output, and the statement is non-empty. This leaves 901 problems and 274,375 submissions. The experiments use the 459 of those 901 for which Qwen3-Coder wrote a program that passed the stored official tests: 4.6% of the source release and 50.9% of the 901, representing 168,715 Python 3 submissions.

Older formulas in the upstream release had been read from page images, so exponents and subscripts were often flattened, for example 10^18 stored as 1018. The cleaning step tries only once to restore incorrect patterns in the problem statements.

The processed dataset contains 459 distinct problems and 168,715 submissions, partitioned into Train, Dev, and Test sets as shown in Table 1. Problems were randomly assigned to partitions, with all submissions associated with a given problem kept in the same partition. This problem-disjoint split prevents submissions for the same problem from appearing in both training and evaluation. The Train set supports development of PTEM, the Dev set supports model selection and calibration, and the held-out Test set is reserved for final evaluation.

Table 1: Codeforces dataset partitions for Tagent A1.

Split	Problems	Submissions	Problems (%)
Train	229	75,049	49.9
Dev	123	52,726	26.8
Test	107	40,940	23.3
Total	459	168,715	100.0

2.3 Success Criteria and Metrics

The primary success criterion is whether the student LLM produces a program that passes the executable tests within the permitted repair-round budget. We measure repair success rate and rounds-to-pass, comparing outcomes across the teacher-memory conditions. Secondary measures include feedback quality, gatekeeper rejection rates, and solution-disclosure violations. We also examine interaction traces and resource consumption, including token usage, latency, and cost. These measures will help determine whether PTEM improves repair outcomes, whether its components contribute independently, and what trade-offs accompany any gains.

3 Methods

3.1 Agent Architecture

The proposed Tagent A1 architecture (Figure 1) comprises three LLM-based agents—a student, a teacher, and a gatekeeper—operating in an iterative Python code-repair loop.

Starting from an incorrect Codeforces submission, the student attempts a repair, which Harbor executes and verifies against executable tests in a Docker-based sandbox. If the tests fail and repair rounds remain, the teacher receives the programming task, the student's code, and permitted test outcomes, then generates feedback intended to guide the student without revealing the solution. The gatekeeper screens the feedback for prohibited solution disclosure, rejecting and requesting regeneration when necessary. The student receives approved feedback and attempts another repair. The process terminates when the tests pass or the maximum repair-round budget is reached.

To approximate student coding behavior, we employed three small language models accessed via OpenRouter: Llama 3.2 1B, Ministral 3B and Llama 3.2 3B. These models were selected for their modest parameter counts and correspondingly limited coding ability, making them plausible proxies for students who can attempt but not reliably solve the target problems. To establish a baseline of unaided problem-solving capacity, we prompted each model to generate solutions for the 224 training-set problems (five independent trials per problem). Table 3 (appendix) reports the results. The models solve only a minority of problems without assistance, providing a baseline against which to evaluate subsequent gains from teacher feedback and PTEM.

The teacher may retrieve relevant prior teaching experience from PTEM, comprising interaction records and distilled lessons. Training episodes generate this experience, while the Dev set supports model selection and calibration without contributing to the final PTEM. Before held-out Test evaluation, PTEM is frozen and versioned. Test interactions do not update memory. Across experimental conditions, the student model and its configuration remain fixed, and paired evaluations use the same

faulty starting submissions, executable tests, and repair-round budget. These controls are intended to isolate the effects of teacher feedback and memory.

The proposed experiments (Table 2) compare a retry-only control with minimal or no substantive teacher feedback, a teacher providing substantive feedback without PTEM, and a teacher using full PTEM (records and lessons). The first comparison assesses whether teacher feedback improves outcomes beyond allowing additional repair attempts; the second isolates the contribution of teacher experience memory. Records-only and lessons-only variants (Section 3.2) further examine the contributions of PTEM's components. The retry-only and no-memory conditions also help calibrate the student model so that it is neither too weak to benefit from feedback nor too strong to require it.

3.2 Persistent Teacher Experience Memory (PTEM)

PTEM stores two complementary forms of teacher experience: *records*, which capture the observed programming mistake, feedback provided, and resulting repair outcome, and *lessons*, which distill generalizable insights about effective feedback from prior teaching interactions. Records preserve evidence of what occurred in individual episodes, while lessons aim to identify feedback strategies that may transfer to other programming problems. Both are derived from training interactions rather than from held-out evaluation cases.

When generating feedback, the PTEM-equipped teacher retrieves relevant records, lessons, or both, according to the experimental condition. Retrieved experience supplements the current programming task, student code, and permitted test outcomes, allowing the teacher to adapt its feedback strategy without directly revealing the repair. The records-only and lessons-only variants isolate the contributions of the two memory components. PTEM is frozen before held-out testing so that evaluation outcomes cannot influence subsequent feedback through memory updates.

3.3 Experimental Design and Ablation Studies

We propose five experimental conditions (Table 2) to isolate the effects of substantive teacher feedback, PTEM, and its two components. All conditions use the same fixed student model, starting submissions, executable tests, and repair-round budget.

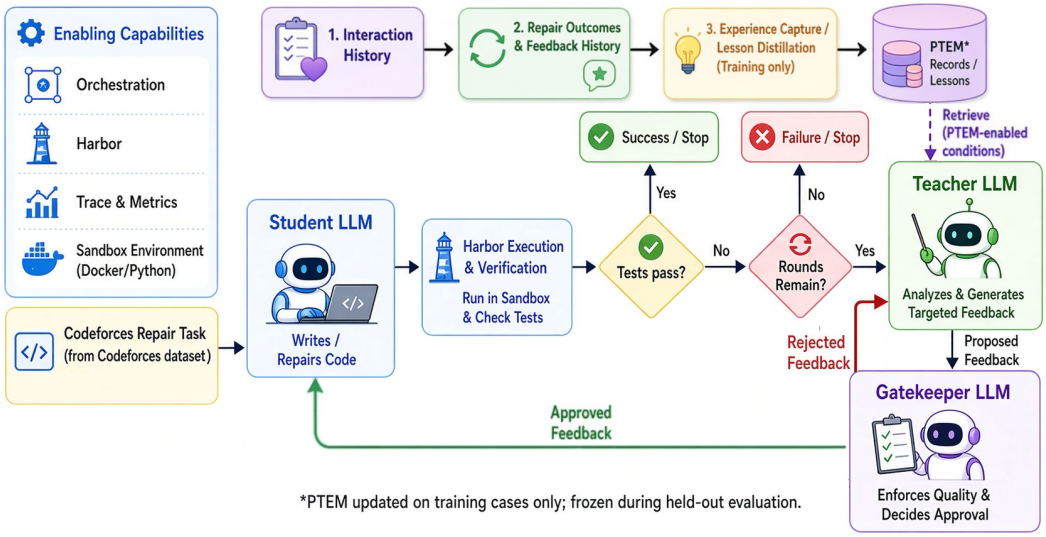


Figure 1: Proposed Tagent A1 architecture. A fixed student LLM iteratively repairs incorrect Codeforces submissions using teacher-generated feedback screened by a gatekeeper. Harbor executes and verifies candidate repairs within a fixed round budget. Training interactions produce teacher experience records and distilled lessons, which the teacher retrieves when PTEM is enabled. PTEM remains frozen during held-out evaluation.

Table 2: Proposed A1 experimental conditions and memory ablations.

Condition	PTEM	Teacher feedback	PTEM records	PTEM lessons
A: Retry-only control	Off	Minimal / none	No	No
B: Teacher baseline	Off	Substantive	No	No
C: Full PTEM	On	Substantive	Yes	Yes
D: Records-only ablation	Partial	Substantive	Yes	No
E: Lessons-only ablation	Partial	Substantive	No	Yes

PTEM = Persistent Teacher Experience Memory. All conditions use the same fixed student model, starting submissions, tests, and repair budget. PTEM is constructed from training interactions and frozen before held-out evaluation.

Comparison of conditions A and B tests whether substantive teacher feedback improves repair outcomes beyond additional attempts without guidance. Comparing B and C isolates the effect of PTEM (RQ1), while comparing B, C, D, and E assesses the individual and combined contributions of records and lessons (RQ2).

The present investigation focuses on stage A1 of the proposed four-stage roadmap (A1–A2–B1–B2), isolating the contribution of teacher experience memory while holding the student fixed. Later stages may investigate richer teacher architectures, student adaptation, and teacher–student co-adaptation.

3.4 Evaluation Protocol and Metrics

We evaluate the five experimental conditions on held-out Codeforces problems using the same fixed student LLM, starting submissions, executable tests, and maximum repair-round budget. PTEM is frozen before evaluation, and test interactions do not update memory. The primary outcome measures are repair success rate (the proportion of attempts producing a program that passes all executable tests within the round budget) and rounds-to-pass for successful attempts. Reporting both measures distinguishes conditions that achieve more successful repairs from those that require fewer rounds among successful cases. Paired comparisons across conditions will assess the effects of teacher feedback and PTEM.

Secondary measures include gatekeeper rejection rates, solution-disclosure violations, and feedback quality, assessed for correctness, relevance, clarity, helpfulness, and pedagogical appropriateness using an LLM judge. For RQ3, human evaluators will compare feedback from the PTEM-equipped and no-memory teachers, subject to feasibility. We will also examine interaction traces to identify failure modes and record resource measures, including token usage, latency, cost, and reliability.

4 Summary of Progress

We have defined the Tagent A1 research questions, architecture, experimental conditions, and evaluation approach. The Codeforces pipeline yields 459 problems and 168,715 Python submissions across Train, Dev, and Test. Flattened notation was corrected where unambiguous. Qwen3-Coder solved each retained problem from its public statement; solutions were verified against official tests.

We have also run the student-only condition in Harbor. Llama 3.2 1B, Llama 3.2 3B, and Ministral 3B each made five attempts at temperature 0.7, with the teacher line ”try again” and no worked solution. On 224 of the 229 training problems, they solved 5.4%, 14.3%, and 38.4%. Student-only baselines were also run on Dev and Test. Ministral 3B is strong enough to make progress and still fails most problems unaided. The smaller Llama models rarely solve a problem even in five attempts.

Remaining work is to integrate the iterative student-teacher-gatekeeper workflow, implement PTEM record capture, lesson distillation, and retrieval, and run the controlled comparisons and memory ablations. We still need to validate the full evaluation procedure, analyze interaction traces and resource costs, and obtain human feedback for RQ3 if feasible.

Remaining risks include solution disclosure, experimental variability, and integration within time and compute constraints. Dataset alignment is complete.

DATA

Table 3: Unaided problem-solving performance of the student-proxy models on the training set.

Model	Problems solved	Passing trials	Passing trials (%)	Mean passes (solved)
Llama 3.2 1B	12/224 (5.36%)	18/1120	1.61%	1.50 of 5
Ministral 3B	86/224 (38.39%)	218/1120	19.46%	2.53 of 5
Llama 3.2 3B	32/224 (14.29%)	61/1120	5.45%	1.91 of 5

REFERENCES

Rishabh Balse, Bharath Valaboju, Shreya Singhal, Jayakrishnan Madathil Warriem, and Prajish Prasad. Investigating the potential of GPT-3 in providing feedback for programming assessments. In *Proceedings of the 2023 Conference on Innovation and Technology in Computer Science Education*, volume 1, pp. 292–298, 2023. doi: 10.1145/3587102.3588852.

Charles Koutcheme, Nicola Dainese, Sami Sarsa, Arto Hellas, Juho Leinonen, Syed Ashraf, and Paul Denny. Evaluating language models for generating and judging programming feedback. In *Proceedings of the 56th ACM Technical Symposium on Computer Science Education*, volume 1, pp. 624–630, 2025. doi: 10.1145/3641554.3701791.

Charles Koutcheme, Juliette Woodrow, and Chris Piech. Aligning small language models for programming feedback: Towards scalable coding support in a massive global course. In *Proceedings of the 57th ACM Technical Symposium on Computer Science Education*, volume 1, pp. 610–616, 2026. doi: 10.1145/3770762.3772539.

Guilherme Penedo, Anton Lozhkov, Hynek Kydlíček, Loubna Ben Allal, Edward Beeching, Agustín Piqueres Lajarín, Quentin Gallouédec, Nathan Habib, Lewis Tunstall, and Leandro von Werra. CodeForces. Hugging Face dataset, 2025. URL `https://huggingface.co/datasets/open-r1/codeforces`.

Sierra Wang, John Mitchell, and Chris Piech. A large scale RCT on effective error messages in CS1. In *Proceedings of the 55th ACM Technical Symposium on Computer Science Education*, volume 1, pp. 1395–1401, 2024. doi: 10.1145/3626252.3630764.

Weixian Xu, Yanzhe Zhang, Zora Zhiruo Wang, Changyu Chen, and Diyi Yang. Sherpa: Teaching llms to teach adaptively, 2026. URL `https://arxiv.org/abs/2610.08778`.

AI USE STATEMENT

During this project, AI tools (including Grok, ChatGPT, Claude, and Claude Code) assisted with the data cleaning, software development and debugging, documentation, drafting, revision, and critical review. Generative AI models were also used to assess programming-problem solvability, identify and propose corrections to problem statements, and evaluate candidate student models and coding tasks. AI-assisted code and research outputs have been reviewed by the authors, who remain responsible for the project's methodology, implementation, results, and conclusions. The experimental teacher, student, and gatekeeper are separate LLM-based components of the proposed system, whose use and evaluation are described in the methodology.

REPRODUCING THE MIDWAY IMPLEMENTATION

The source code and supporting materials for the Tagent midway implementation are maintained in a public GitHub repository: `https://github.com/chesnay/CS329Z_midway`

Reviewers can clone the repository using:

```
git clone https://github.com/chesnay/CS329Z_midway.git
cd CS329Z_midway
```

Run every later command from `CS329Z_midway`, the directory that contains `pyproject.toml`.

Environment and Dependencies.

The commands were prepared for macOS and Linux. The repository requires Python 3.12 and uv. A Harbor trial also requires Docker. Install Docker from the Mac instructions or the Linux instructions.

Launch Docker Desktop (Mac/Windows) and wait until it reports the engine is running. From Terminal you can also do:

```
open -a Docker
```

Confirm it's ready:

```
docker info
```

Then run the verification pull the ubuntu:22.04 image. That download checks that Docker can fetch an image. The demo tasks do not use this image. Harbor builds their image from 'environment/Dockerfile' when you run a task.

```
docker pull ubuntu:22.04
```

The demo tasks do not use that image. Each task builds its own image from `environment/Dockerfile`, which starts from `python:3.12-slim-bookworm`. From the repository root, install the pinned dependencies and check that Harbor can reach Docker:

```
uv sync
uv run harbor --version
uv run harbor --help
docker version
```

`uv sync` installs Harbor 0.18.0 and the libraries that read the Parquet tables. `harbor --version` should print `0.18.0`. `docker version` should print both a Client section and a Server section. A missing Server section means the Docker client cannot reach the Docker server.

Configuration.

A live model call needs an OpenRouter inference key. Reading the stored dataset does not. Copy the template and set `OPENROUTER_API_KEY`. The template also names `GEMINI_API_KEY`. A Gemini key does not run the OpenRouter model identifiers in this repository. Leave it blank unless Harbor is pointed at a Gemini model. Credentials are not included in the repository. `.env` is listed in `.gitignore`.

```
cp .env.example .env
```

Harbor reads that file only when a command passes `--env-file .env`. The student agent is not installed as a package. Every model command below sets `PYTHONPATH=src` so Harbor can import the agent from `src/tagent/agents/rewrite_student.py`.

Execution.

The offline checks need no API key and no Docker. They test the selection rule, print the stored scores, list the Train problems, list the submissions for one problem, and print that problem's small-model scores:

```
uv run pytest
uv run python scripts/summarize_clean_codeforces.py
uv run python scripts/list_clean_codeforces.py --split train
uv run python scripts/list_clean_codeforces.py --split dev --problem 26/A
uv run python scripts/show_clean_runs.py --split dev --problem 26/A
```

Replace `train` with `dev` or `test` to list another split. Add `--source 101817015` to the `26/A` command to print that one program. The submission identifier has to belong to the problem.

The evaluation example is one attempt of Llama 3.2 1B on Codeforces `630/A`, starting from rejected submission `230389330`. It spends OpenRouter credits. Start Docker first.

```
PYTHONPATH=src uv run harbor run \
  --path data/example_code/harbor_tasks/demo/train/630_A__230389330 \
  --agent tagent.agents.rewrite_student:RewriteStudent \
  --model openrouter/meta-llama/llama-3.2-1b-instruct \
  --env-file .env \
  --yes \
  --n-attempts 1 \
  --agent-kwarg temperature=0.7 \
  --job-name demo__llama-3.2-1b__630a \
  --jobs-dir data/example_code/harbor_jobs
```

The same three tasks, `630/A`, `630/C`, and `825/A`, can be run with any of the four models. Each command uses one attempt at temperature 0.7. The reported small-model rates used five attempts. Reusing a `--job-name` resumes the old job, so each command has its own name.

```
PYTHONPATH=src uv run harbor run \
  --path data/example_code/harbor_tasks/demo/train \
  --agent tagent.agents.rewrite_student:RewriteStudent \
  --model openrouter/meta-llama/llama-3.2-1b-instruct \
  --env-file .env --yes --n-attempts 1 \
  --agent-kwarg temperature=0.7 \
  --job-name demo__llama-3.2-1b \
  --jobs-dir data/example_code/harbor_jobs

PYTHONPATH=src uv run harbor run \
  --path data/example_code/harbor_tasks/demo/train \
  --agent tagent.agents.rewrite_student:RewriteStudent \
  --model openrouter/mistralai/ministral-3b-2512 \
  --env-file .env --yes --n-attempts 1 \
  --agent-kwarg temperature=0.7 \
  --job-name demo__ministral-3b \
  --jobs-dir data/example_code/harbor_jobs

PYTHONPATH=src uv run harbor run \
  --path data/example_code/harbor_tasks/demo/train \
  --agent tagent.agents.rewrite_student:RewriteStudent \
  --model openrouter/meta-llama/llama-3.2-3b-instruct \
  --env-file .env --yes --n-attempts 1 \
  --agent-kwarg temperature=0.7 \
  --job-name demo__llama-3.2-3b \
  --jobs-dir data/example_code/harbor_jobs

PYTHONPATH=src uv run harbor run \
  --path data/example_code/harbor_tasks/demo/train \
  --agent tagent.agents.rewrite_student:RewriteStudent \
  --model openrouter/qwen/qwen3-coder \
  --env-file .env --yes --n-attempts 1 \
  --agent-kwarg temperature=0.7 \
  --job-name demo__qwen3-coder \
  --jobs-dir data/example_code/harbor_jobs
```

Qwen3-Coder selected the 459 problems from the public statement only, with up to three tries and no rejected program. This repository cannot replay that selection. On these three tasks the command

above uses the same repair agent as the small models, so the model also sees the rejected submission and the line "try again." Raising `--n-attempts` to 3 matches the selection protocol and needs a new job name if `demo__qwen3-coder` has already run.

The three demo tasks also contain an accepted program. Grading that program does not call a model and does not read `.env`:

```
uv run harbor run \
  --path data/example_code/harbor_tasks/demo/train/630_A__230389330 \
  --agent oracle \
  --n-attempts 1 --n-concurrent 1 \
  --job-name demo-oracle \
  --jobs-dir data/example_code/harbor_jobs \
  --yes
```

`scripts/run_rest_train.py` built the full Train, Dev, and Test jobs in the original study. Its case files, split lists, and unfiltered official subset are not in this repository. Do not use it to reproduce the table. The saved scores are already in `data/example_code/harbor_jobs/`.

Expected Outputs.

`uv run pytest` passes 18 tests. `summarize_clean_codeforces.py` prints the stored dataset and the small-model results. On the clean Train set, 224 of 229 problems were run for five attempts at temperature 0.7: Llama 3.2 1B solved 12/224 problems (18/1120 trials), Ministral 3B solved 86/224 (218/1120), and Llama 3.2 3B solved 32/224 (61/1120). The same script prints Dev, 121 of 123 problems, and Test, 106 of 107. A cell such as `2/5` means two of five attempts passed. `not run` means the problem was not given to the small models. `show_clean_runs.py --split dev --problem 26/A` prints `0/5`, `2/5`, and `3/5` for the three small models.

A Harbor run writes `data/example_code/harbor_jobs/<job-name>/result.json` and, inside each trial, `verifier/reward.txt`. That file is `1` when every official test passed and `0` otherwise. The terminal table reports the same reward. A reward of `0` for Llama 3.2 1B on these problems is a normal outcome, not a failed installation. The reference-solution run on `630/A` should report a reward of `1.0`. Finished logs can be browsed with `uv run harbor view data/example_code/harbor_jobs`, which serves a page on `http://127.0.0.1:8080`. The per-trial logs of the original study are not in the repository. A new run writes its own log locally. Do not commit that log.